

Kyle Lin
ESE 358 Project 4

Input, output, list of steps followed, and source code

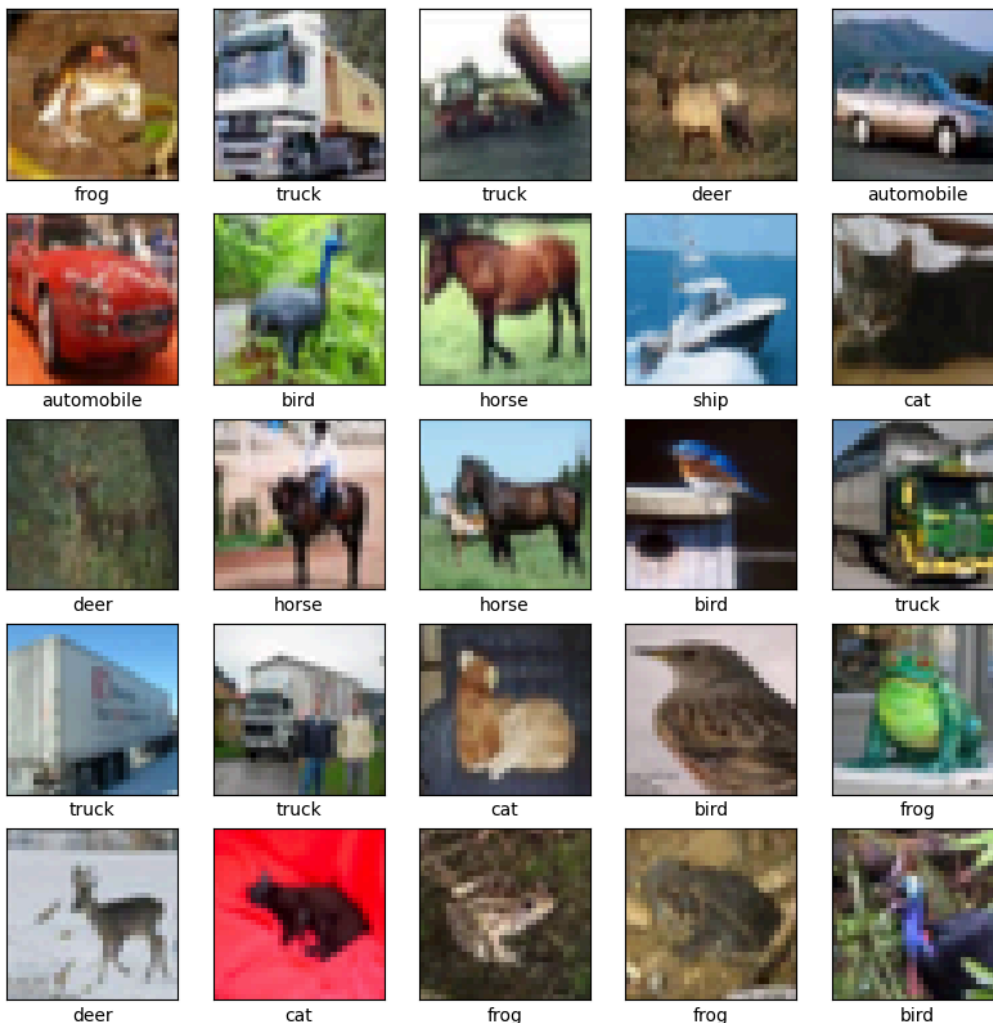
Convolutional Neural Network (CNN)

Input:

```
import tensorflow as tf
```

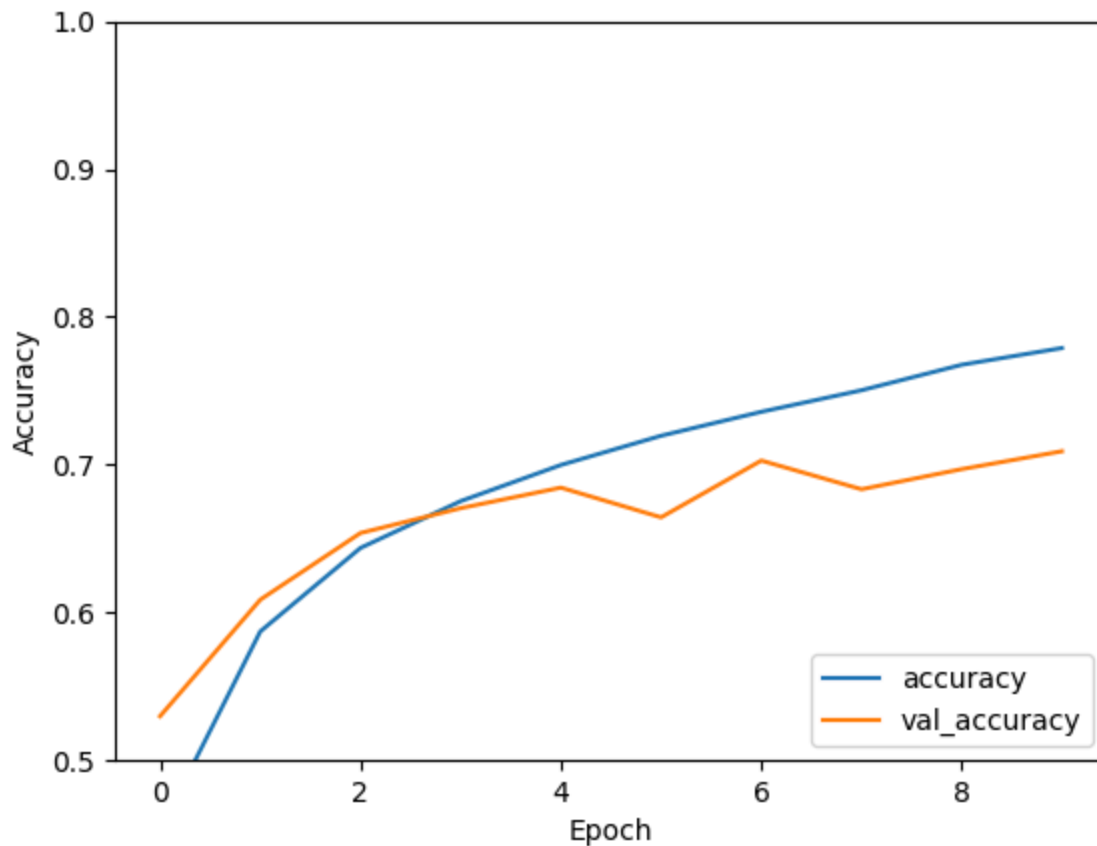
Output:

```
PS C:\Users\Kyle\VSCode\Python\ESE 358 Project 4> & C:/Users/Kyle/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/Kyle/VSCode/Python/ESE 358 Project 4/cnn.py"
Traceback (most recent call last):
  File "c:/Users/Kyle/VSCode/Python/ESE 358 Project 4/cnn.py", line 1, in <module>
    import tensorflow as tf
ModuleNotFoundError: No module named 'tensorflow'
PS C:\Users\Kyle\VSCode\Python\ESE 358 Project 4> & C:/Users/Kyle/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/Kyle/VSCode/Python/ESE 358 Project 4/cnn.py"
2025-11-14 00:25:18.860279: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-11-14 00:25:24.199635: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
PS C:\Users\Kyle\VSCode\Python\ESE 358 Project 4> & C:/Users/Kyle/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/Kyle/VSCode/Python/ESE 358 Project 4/cnn.py"
2025-11-14 00:25:34.956316: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-11-14 00:25:36.953269: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 28s 0us/step
```



Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_1 (Conv2D)	(None, 13, 13, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)	0
conv2d_2 (Conv2D)	(None, 4, 4, 64)	36,928
flatten (Flatten)	(None, 1024)	0
dense (Dense)	(None, 64)	65,600
dense_1 (Dense)	(None, 10)	650

313/313 - 1s - 2ms/step - accuracy: 0.7209 - loss: 0.8425



0.7208999991416931

Steps

1. Import the libraries: TensorFlow / Keras and Matplotlib
2. Load the CIFAR-10 dataset using datasets. Get training and test images and their labels.
3. Normalize all image pixel values to the range 0–1 by dividing the image arrays by 255.0.
4. Define the list of class names
5. Visualize the data by plotting the first 25 training images in a 5 by 5 grid and labeling each image with its corresponding class name.
6. Build the convolutional base of the model using a Sequential network with:
 - a. A Conv2D layer with 32 filters and a 3 by 3 kernel, ReLU activation, and input shape 32×32×3.
 - b. A MaxPooling2D layer with a pool size of 2 by 2
 - c. A Conv2D layer with 64 filters and a 3 by 3 kernel, ReLU activation.
 - d. A second MaxPooling2D layer with a pool size of 2 by 2.
 - e. A third Conv2D layer with 64 filters and a 3 by 3 kernel, ReLU activation.

7. Add the classifier on top of the convolutional base by:
 - a. Flattening the 4 by 4 by 64 output with a Flatten layer.
 - b. Adding a Dense layer with 64 units and ReLU activation.
 - c. Adding a final Dense layer with 10 units.
8. Compile the model with the Adam optimizer, the sparse categorical cross-entropy loss, and accuracy as the evaluation metric.
9. Train the model for 10 epochs using the training images and labels, and monitor validation performance using the test images and labels.
10. Evaluate the trained model on the test set to obtain final test loss and test accuracy, and plot the training and validation accuracy curves across epochs.

Source Code

```
import tensorflow as tf

from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt

(train_images, train_labels), (test_images, test_labels) = datasets.cifar10.load_data()

# Normalize pixel values to be between 0 and 1
train_images, test_images = train_images / 255.0, test_images / 255.0

class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']

plt.figure(figsize=(10,10))
for i in range(25):
    plt.subplot(5,5,i+1)
    plt.xticks([ ])
    plt.yticks([ ])
    plt.grid(False)
    plt.imshow(train_images[i])
    # The CIFAR labels happen to be arrays,
    # which is why you need the extra index
    plt.xlabel(class_names[train_labels[i][0]])
plt.show()

model = models.Sequential()
```

```
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))

model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10))

model.summary()

model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])

history = model.fit(train_images, train_labels, epochs=10,
                    validation_data=(test_images, test_labels))

plt.plot(history.history['accuracy'], label='accuracy')
plt.plot(history.history['val_accuracy'], label = 'val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.ylim([0.5, 1])
plt.legend(loc='lower right')
plt.show()

test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=2)

print(test_acc)
```

Input, output, list of steps followed, and source code

Transfer learning and fine-tuning

Output

(32, 5, 5, 1280)
Model: "mobilenetv2_1.00_160"

Layer (type)	Output Shape	Param #	Connected to
input_layer_1 (InputLayer)	(None, 160, 160, 3)	0	-
Conv1 (Conv2D)	(None, 80, 80, 32)	864	input_layer_1[0][0]
bn_Conv1 (BatchNormalization)	(None, 80, 80, 32)	128	Conv1[0][0]
Conv1_relu (ReLU)	(None, 80, 80, 32)	0	bn_Conv1[0][0]
expanded_conv_depthwise (DepthwiseConv2D)	(None, 80, 80, 32)	288	Conv1_relu[0][0]
expanded_conv_depthwise_BN (BatchNormalization)	(None, 80, 80, 32)	128	expanded_conv_depthwise[0][0]
expanded_conv_depthwise_relu (ReLU)	(None, 80, 80, 32)	0	expanded_conv_depthwise_BN[0][0]
expanded_conv_project (Conv2D)	(None, 80, 80, 16)	512	expanded_conv_depthwise_relu[0][0]
expanded_conv_project_BN (BatchNormalization)	(None, 80, 80, 16)	64	expanded_conv_project[0][0]

More output, but can't fit

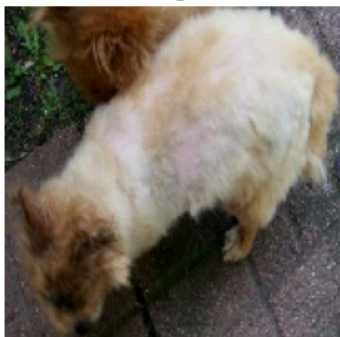
Total params: 2,257,984 (8.61 MB)
Trainable params: 0 (0.00 B)
Non-trainable params: 2,257,984 (8.61 MB)
(32, 1280)
(32, 1)

Model: "functional_1"

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 160, 160, 3)	0
sequential (Sequential)	(None, 160, 160, 3)	0
true_divide (TrueDivide)	(None, 160, 160, 3)	0
subtract (Subtract)	(None, 160, 160, 3)	0
mobilenetv2_1.00_160 (Functional)	(None, 5, 5, 1280)	2,257,984
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dropout (Dropout)	(None, 1280)	0
dense (Dense)	(None, 1)	1,281

Non-trainable params: 2,257,984 (8.61 MB)
26/26 ————— 7s 179ms/step - accuracy: 0.3614 - loss: 0.8797
initial loss: 0.88
initial accuracy: 0.36
Epoch 1/10
63/63 ————— 22s 270ms/step - accuracy: 0.5380 - loss: 0.7668 - val_accuracy: 0.7141 - val_loss: 0.5909
Epoch 2/10
63/63 ————— 21s 278ms/step - accuracy: 0.7125 - loss: 0.5547 - val_accuracy: 0.8564 - val_loss: 0.4284
Epoch 3/10
63/63 ————— 18s 278ms/step - accuracy: 0.7965 - loss: 0.4528 - val_accuracy: 0.9047 - val_loss: 0.3354
Epoch 4/10
63/63 ————— 18s 294ms/step - accuracy: 0.8450 - loss: 0.3800 - val_accuracy: 0.9319 - val_loss: 0.2725
Epoch 5/10
63/63 ————— 16s 247ms/step - accuracy: 0.8785 - loss: 0.3195 - val_accuracy: 0.9418 - val_loss: 0.2260
Epoch 6/10
63/63 ————— 16s 249ms/step - accuracy: 0.8870 - loss: 0.2949 - val_accuracy: 0.9455 - val_loss: 0.2051
Epoch 7/10
63/63 ————— 15s 236ms/step - accuracy: 0.9020 - loss: 0.2675 - val_accuracy: 0.9517 - val_loss: 0.1807
Epoch 8/10
63/63 ————— 17s 275ms/step - accuracy: 0.9030 - loss: 0.2572 - val_accuracy: 0.9579 - val_loss: 0.1622
Epoch 9/10
63/63 ————— 16s 253ms/step - accuracy: 0.9055 - loss: 0.2354 - val_accuracy: 0.9653 - val_loss: 0.1441
Epoch 10/10
63/63 ————— 16s 262ms/step - accuracy: 0.9215 - loss: 0.2110 - val_accuracy: 0.9641 - val_loss: 0.1447

dogs



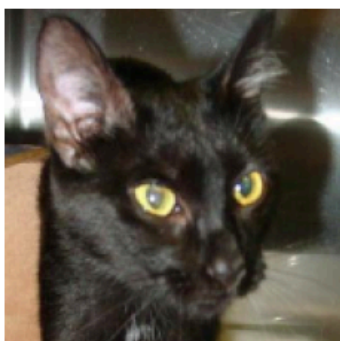
cats



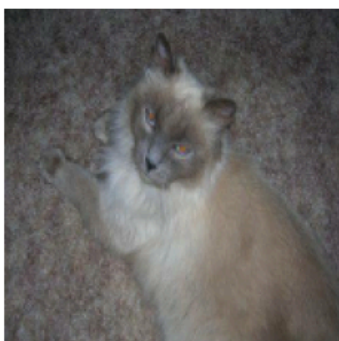
cats



cats



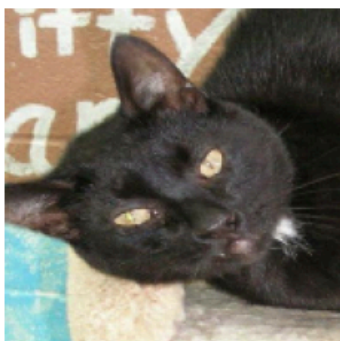
cats



cats



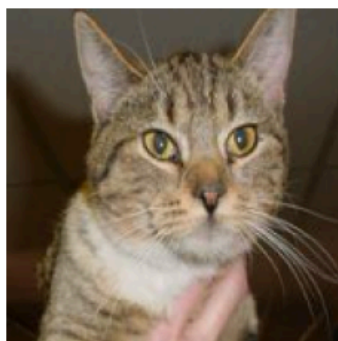
cats

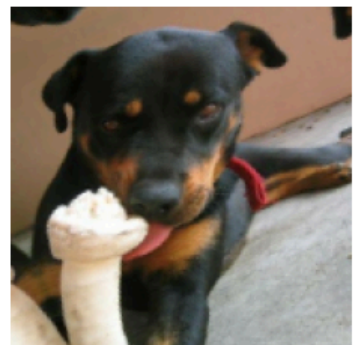
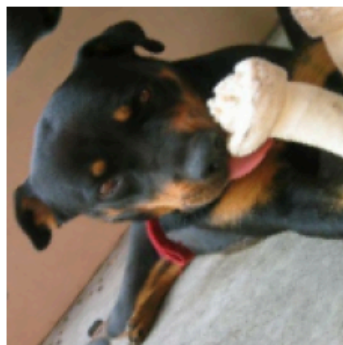
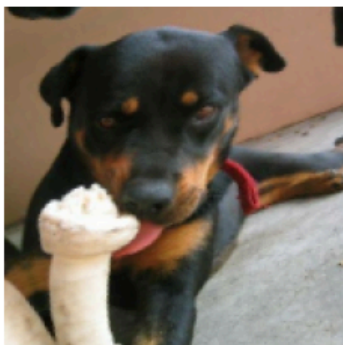
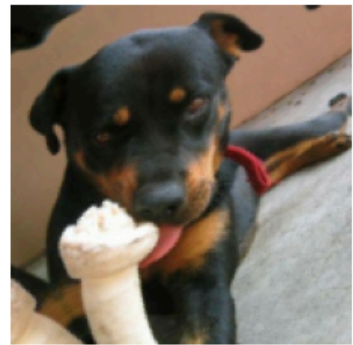
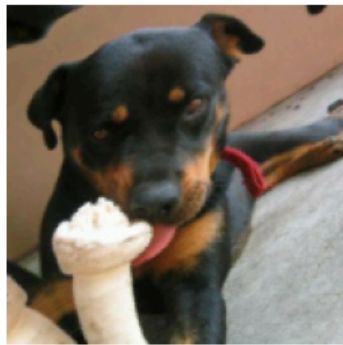
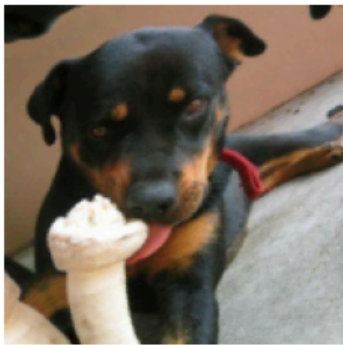
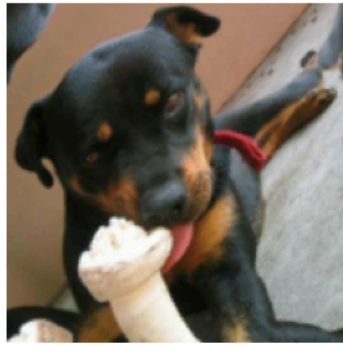
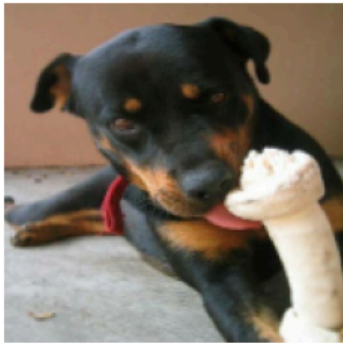


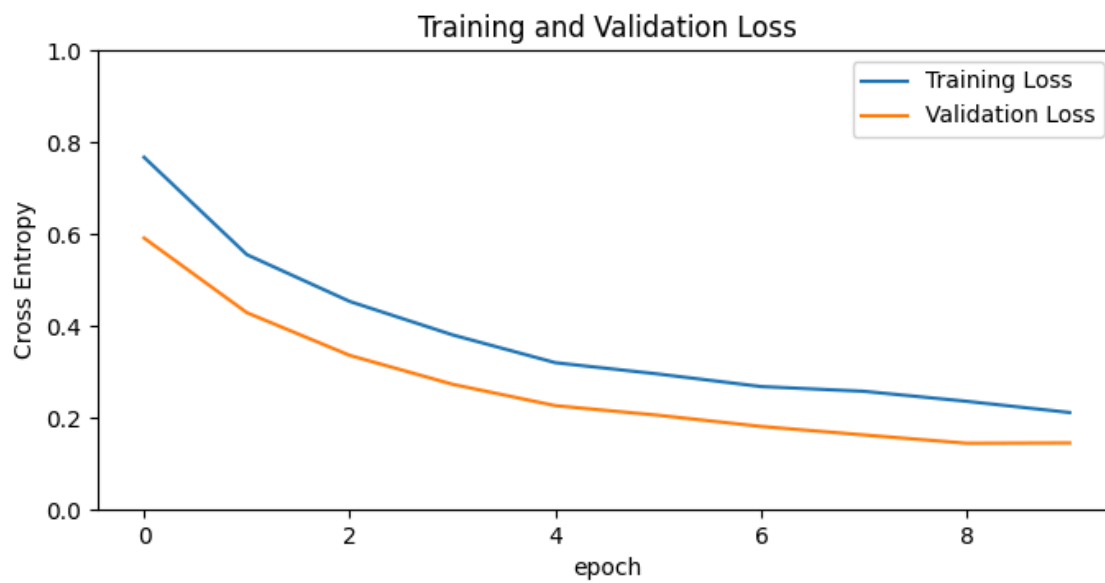
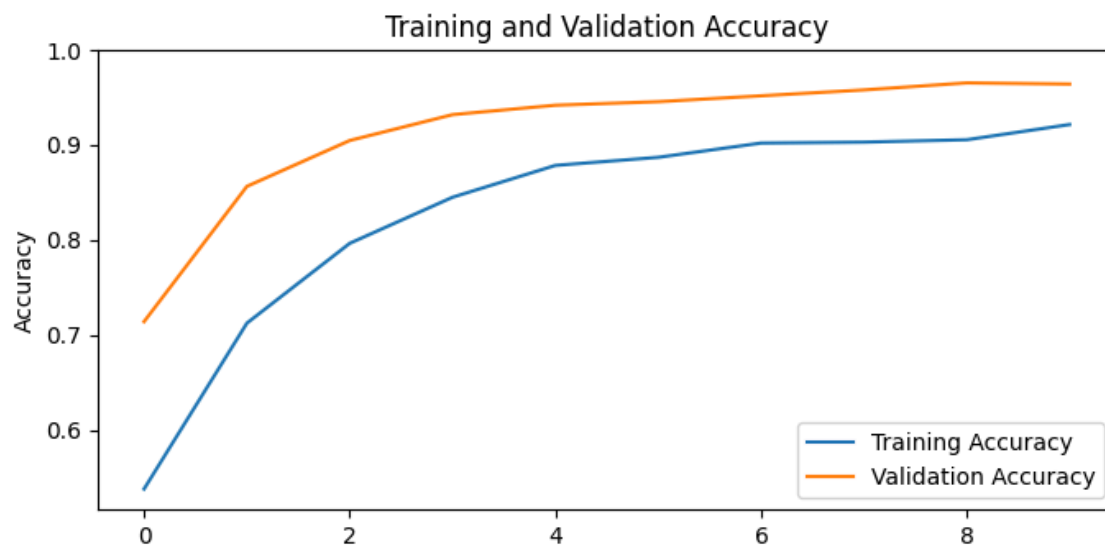
dogs



cats

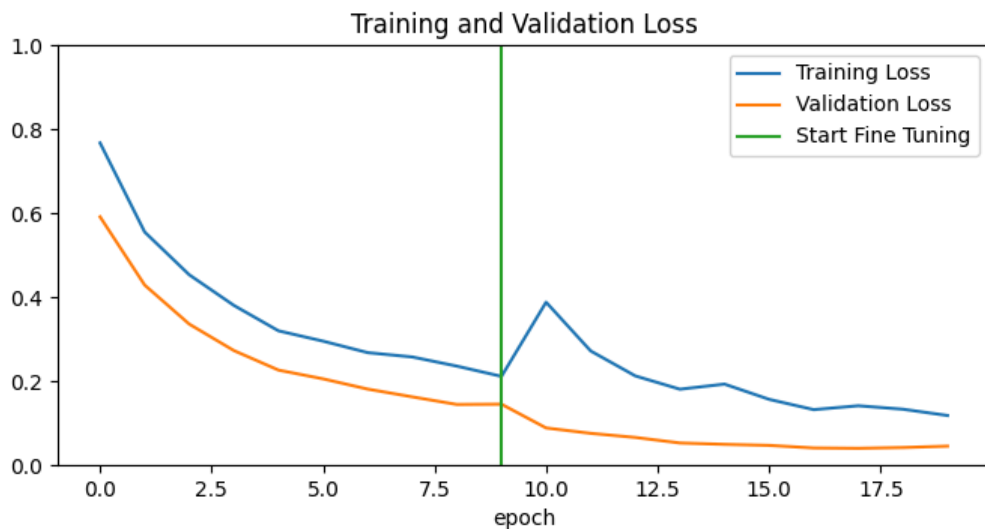
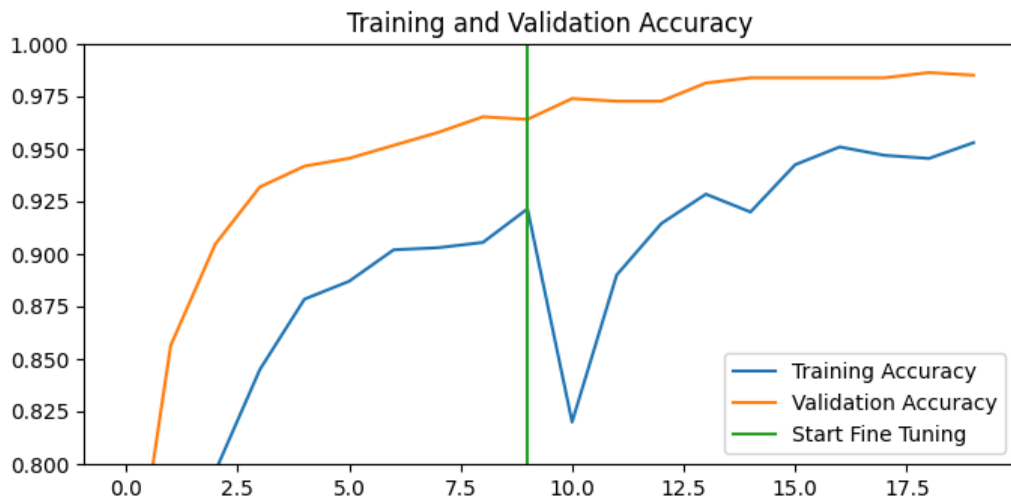






Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 160, 160, 3)	0
sequential (Sequential)	(None, 160, 160, 3)	0
true_divide (TrueDivide)	(None, 160, 160, 3)	0
subtract (Subtract)	(None, 160, 160, 3)	0
mobilenetv2_1.00_160 (Functional)	(None, 5, 5, 1280)	2,257,984
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dropout (Dropout)	(None, 1280)	0
dense (Dense)	(None, 1)	1,281

Total params: 2,259,265 (8.62 MB)
 Trainable params: 1,862,721 (7.11 MB)
 Non-trainable params: 396,544 (1.51 MB)



```

Epoch 11/20
63/63 ————— 25s 295ms/step - accuracy: 0.8200 - loss: 0.3877 - val_accuracy: 0.9740 - val_loss: 0.0882
Epoch 12/20
63/63 ————— 19s 297ms/step - accuracy: 0.8900 - loss: 0.2715 - val_accuracy: 0.9728 - val_loss: 0.0754
Epoch 13/20
63/63 ————— 25s 398ms/step - accuracy: 0.9145 - loss: 0.2121 - val_accuracy: 0.9728 - val_loss: 0.0656
Epoch 14/20
63/63 ————— 20s 309ms/step - accuracy: 0.9285 - loss: 0.1804 - val_accuracy: 0.9814 - val_loss: 0.0524
Epoch 15/20
63/63 ————— 25s 399ms/step - accuracy: 0.9200 - loss: 0.1926 - val_accuracy: 0.9839 - val_loss: 0.0492
Epoch 16/20
63/63 ————— 24s 387ms/step - accuracy: 0.9425 - loss: 0.1563 - val_accuracy: 0.9839 - val_loss: 0.0467
Epoch 17/20
63/63 ————— 25s 394ms/step - accuracy: 0.9510 - loss: 0.1318 - val_accuracy: 0.9839 - val_loss: 0.0405
Epoch 18/20
63/63 ————— 25s 392ms/step - accuracy: 0.9470 - loss: 0.1410 - val_accuracy: 0.9839 - val_loss: 0.0397
Epoch 19/20
63/63 ————— 25s 392ms/step - accuracy: 0.9455 - loss: 0.1328 - val_accuracy: 0.9864 - val_loss: 0.0416
Epoch 20/20
63/63 ————— 24s 388ms/step - accuracy: 0.9530 - loss: 0.1179 - val_accuracy: 0.9851 - val_loss: 0.0448

```

```

6/6 ————— 1s 166ms/step - accuracy: 0.9844 - loss: 0.0497
Test accuracy : 0.984375
Predictions:
[0 1 1 1 0 0 1 0 0 1 0 1 1 0 1 1 0 0 0 0 1 1 1 0 0 0 0 1 0 0 1 0]
Labels:
[0 1 1 1 0 0 1 0 0 1 1 1 1 0 1 1 0 0 0 0 1 1 1 0 0 0 0 1 1 0 1 0]

```

Steps

1. Data download
 - a. Download the Cats vs Dogs dataset.
 - b. Extract it into train/ and validation/ directories with subfolders per class.
2. Configure the dataset for performance
 - a. Load images with `image_dataset_from_directory`
 - b. Use `batch_size` and `image_size`
 - c. Cache, shuffle, and prefetch the datasets for speed.
3. Use data augmentation
 - a. Create a Sequential augmentation model
 - b. Apply it to the input images during training.
4. Rescale pixel values

- a. Add a Rescaling layer (or equivalent) so inputs are in $[0, 1]$ or the range expected by the pre-trained model.
 - b. 2. Create the base model from the pre-trained convnet
5. Load pre-trained model
 - a. Use a convnet like `tf.keras.applications.MobileNetV2` with `include_top=False`.
 - b. Set `input_shape` to match your images.
 - c. Use `weights='imagenet'` to get pretrained weights.
6. Wrap it as a feature extractor
 - a. Feed images through the base model to get feature maps.
 - b. Optionally add a `GlobalAveragePooling2D` after the base model.
 - c. Feature extraction (first training phase)
7. Freeze the convolutional base
 - a. Set `base_model.trainable = False` so its weights don't change.
8. Add a classification head

On top of the base model outputs, add:

 - i. `GlobalAveragePooling2D()`
 - ii. Optional Dropout.
9. Compile the model
 - a. Use an optimizer like adam.
 - b. Use appropriate loss
 - c. Track metrics such as accuracy.
10. Train the model (feature extraction)
 - a. Train for several epochs with the frozen base.
 - b. Only the new classification layers' weights update.
11. Plot learning curves
 - a. Plot training vs validation loss/accuracy to see how the transfer-learned model performs.
 - b. 4. Fine-tuning (second training phase)
12. Unfreeze the top layers of the base model
13. Re-compile the model (with a lower learning rate)
 - a. Use a smaller learning rate to avoid destroying pre-trained weights
14. Continue training the model
 - a. Train again for more epochs
 - b. Now both the classification head and the unfrozen top conv layers are updated.
15. Evaluation and prediction
 - a. Evaluate on the validation/test set to measure final performance.
 - b. Use `model.predict` on sample images to see class probabilities/labels.

Source Code

```
import matplotlib.pyplot as plt
import numpy as np
import os
import tensorflow as tf
os.environ["PATH"] += os.pathsep + r"C:\Users\Kyle\Downloads\Graphviz-14.0.2-win64\bin"
from tensorflow.keras.utils import plot_model

_URL = 'https://storage.googleapis.com/mledu-datasets/cats_and_dogs_filtered.zip'
path_to_zip = tf.keras.utils.get_file('cats_and_dogs.zip', origin=_URL, extract=True)
PATH = os.path.join(os.path.dirname(path_to_zip), 'cats_and_dogs_filtered')

train_dir = os.path.join(PATH, 'train')
validation_dir = os.path.join(PATH, 'validation')

BATCH_SIZE = 32
IMG_SIZE = (160, 160)

train_dataset = tf.keras.utils.image_dataset_from_directory(train_dir,
                                                            shuffle=True,
                                                            batch_size=BATCH_SIZE,
                                                            image_size=IMG_SIZE)

validation_dataset = tf.keras.utils.image_dataset_from_directory(validation_dir,
                                                                shuffle=True,
                                                                batch_size=BATCH_SIZE,
                                                                image_size=IMG_SIZE)

class_names = train_dataset.class_names

plt.figure(figsize=(10, 10))
for images, labels in train_dataset.take(1):
    for i in range(9):
        ax = plt.subplot(3, 3, i + 1)
        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title(class_names[labels[i]])
        plt.axis("off")

val_batches = tf.data.experimental.cardinality(validation_dataset)
```

```

test_dataset = validation_dataset.take(val_batches // 5)
validation_dataset = validation_dataset.skip(val_batches // 5)

print('Number of validation batches: %d' % tf.data.experimental.cardinality(validation_dataset))
print('Number of test batches: %d' % tf.data.experimental.cardinality(test_dataset))

AUTOTUNE = tf.data.AUTOTUNE

train_dataset = train_dataset.prefetch(buffer_size=AUTOTUNE)
validation_dataset = validation_dataset.prefetch(buffer_size=AUTOTUNE)
test_dataset = test_dataset.prefetch(buffer_size=AUTOTUNE)

data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip('horizontal'),
    tf.keras.layers.RandomRotation(0.2),
])

for image, _ in train_dataset.take(1):
    plt.figure(figsize=(10, 10))
    first_image = image[0]
    for i in range(9):
        ax = plt.subplot(3, 3, i + 1)
        augmented_image = data_augmentation(tf.expand_dims(first_image, 0))
        plt.imshow(augmented_image[0] / 255)
        plt.axis('off')

preprocess_input = tf.keras.applications.mobilenet_v2.preprocess_input

rescale = tf.keras.layers.Rescaling(1./127.5, offset=-1)

# Create the base model from the pre-trained model MobileNet V2
IMG_SHAPE = IMG_SIZE + (3,)
base_model = tf.keras.applications.MobileNetV2(input_shape=IMG_SHAPE,
                                                include_top=False,
                                                weights='imagenet')

image_batch, label_batch = next(iter(train_dataset))
feature_batch = base_model(image_batch)
print(feature_batch.shape)

```

```

base_model.trainable = False

# Let's take a look at the base model architecture
base_model.summary()

global_average_layer = tf.keras.layers.GlobalAveragePooling2D()
feature_batch_average = global_average_layer(feature_batch)
print(feature_batch_average.shape)

prediction_layer = tf.keras.layers.Dense(1, activation='sigmoid')
prediction_batch = prediction_layer(feature_batch_average)
print(prediction_batch.shape)

inputs = tf.keras.Input(shape=(160, 160, 3))
x = data_augmentation(inputs)
x = preprocess_input(x)
x = base_model(x, training=False)
x = global_average_layer(x)
x = tf.keras.layers.Dropout(0.2)(x)
outputs = prediction_layer(x)
model = tf.keras.Model(inputs, outputs)

model.summary()

len(model.trainable_variables)

tf.keras.utils.plot_model(model, show_shapes=True)

base_learning_rate = 0.0001
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=base_learning_rate),
              loss=tf.keras.losses.BinaryCrossentropy(),
              metrics=[tf.keras.metrics.BinaryAccuracy(threshold=0.5, name='accuracy')])

initial_epochs = 10

loss0, accuracy0 = model.evaluate(validation_dataset)

print("initial loss: {:.2f}".format(loss0))
print("initial accuracy: {:.2f}".format(accuracy0))

```



```

history = model.fit(train_dataset,
                    epochs=initial_epochs,
                    validation_data=validation_dataset)

acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

loss = history.history['loss']
val_loss = history.history['val_loss']

plt.figure(figsize=(8, 8))
plt.subplot(2, 1, 1)
plt.plot(acc, label='Training Accuracy')
plt.plot(val_acc, label='Validation Accuracy')
plt.legend(loc='lower right')
plt.ylabel('Accuracy')
plt.ylim([min(plt.ylim()), 1])
plt.title('Training and Validation Accuracy')

plt.subplot(2, 1, 2)
plt.plot(loss, label='Training Loss')
plt.plot(val_loss, label='Validation Loss')
plt.legend(loc='upper right')
plt.ylabel('Cross Entropy')
plt.ylim([0, 1.0])
plt.title('Training and Validation Loss')
plt.xlabel('epoch')
plt.show()

base_model.trainable = True

# Let's take a look to see how many layers are in the base model
print("Number of layers in the base model: ", len(base_model.layers))

# Fine-tune from this layer onwards
fine_tune_at = 100

# Freeze all the layers before the `fine_tune_at` layer
for layer in base_model.layers[:fine_tune_at]:
    layer.trainable = False

```

```

model.compile(loss=tf.keras.losses.BinaryCrossentropy(),
              optimizer = tf.keras.optimizers.RMSprop(learning_rate=base_learning_rate/10),
              metrics=[tf.keras.metrics.BinaryAccuracy(threshold=0.5, name='accuracy')])

model.summary()

len(model.trainable_variables)

fine_tune_epochs = 10
total_epochs = initial_epochs + fine_tune_epochs

history_fine = model.fit(train_dataset,
                        epochs=total_epochs,
                        initial_epoch=len(history.epoch),
                        validation_data=validation_dataset)

acc += history_fine.history['accuracy']
val_acc += history_fine.history['val_accuracy']

loss += history_fine.history['loss']
val_loss += history_fine.history['val_loss']

plt.figure(figsize=(8, 8))
plt.subplot(2, 1, 1)
plt.plot(acc, label='Training Accuracy')
plt.plot(val_acc, label='Validation Accuracy')
plt.ylim([0.8, 1])
plt.plot([initial_epochs-1,initial_epochs-1],
         plt.ylim(), label='Start Fine Tuning')
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')

plt.subplot(2, 1, 2)
plt.plot(loss, label='Training Loss')
plt.plot(val_loss, label='Validation Loss')
plt.ylim([0, 1.0])
plt.plot([initial_epochs-1,initial_epochs-1],
         plt.ylim(), label='Start Fine Tuning')
plt.legend(loc='upper right')

```

```
plt.title('Training and Validation Loss')
plt.xlabel('epoch')
plt.show()

loss, accuracy = model.evaluate(test_dataset)
print('Test accuracy :', accuracy)

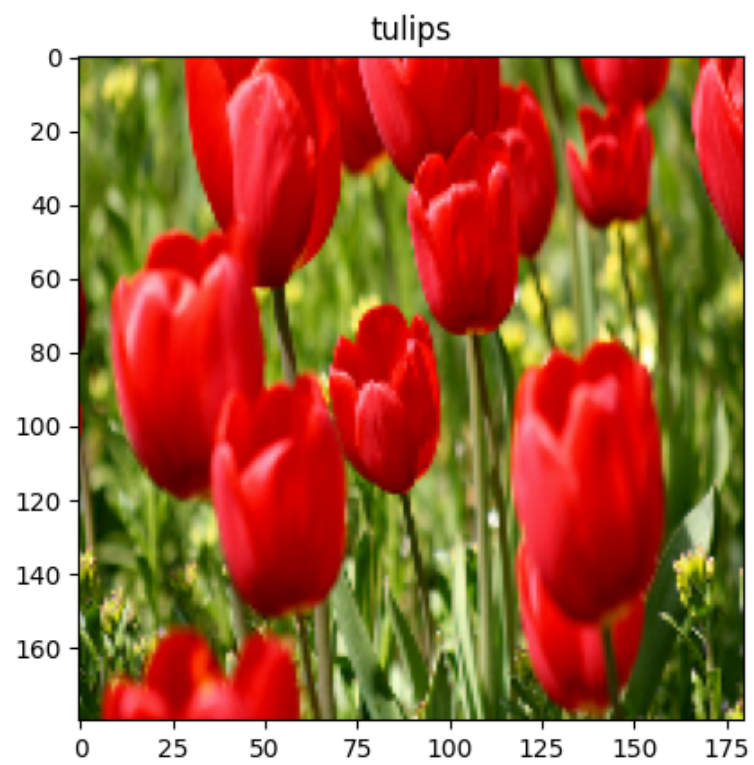
# Retrieve a batch of images from the test set
image_batch, label_batch = test_dataset.as_numpy_iterator().next()
predictions = model.predict_on_batch(image_batch).flatten()
predictions = tf.where(predictions < 0.5, 0, 1)

print('Predictions:\n', predictions.numpy())
print('Labels:\n', label_batch)

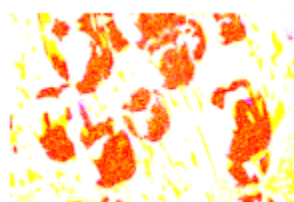
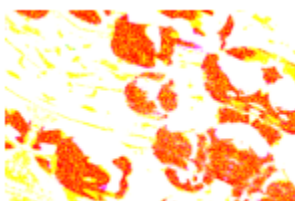
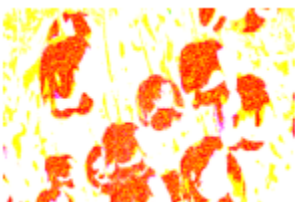
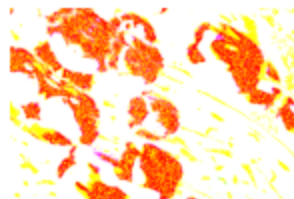
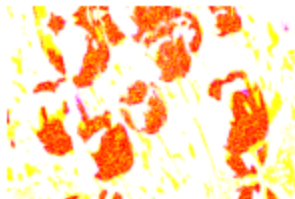
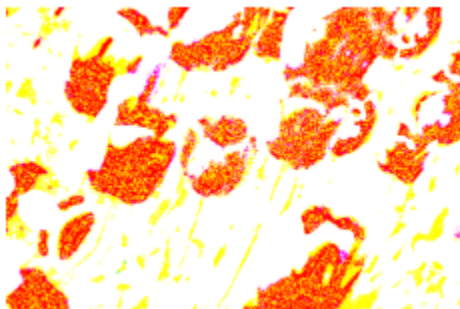
plt.figure(figsize=(10, 10))
for i in range(9):
    ax = plt.subplot(3, 3, i + 1)
    plt.imshow(image_batch[i].astype("uint8"))
    plt.title(class_names[predictions[i]])
    plt.axis("off")
```

Data Augmentation

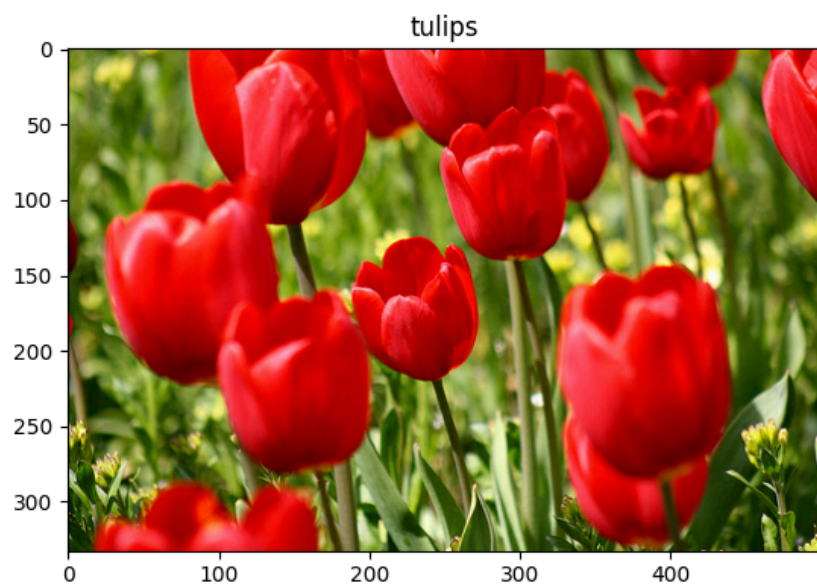
Output

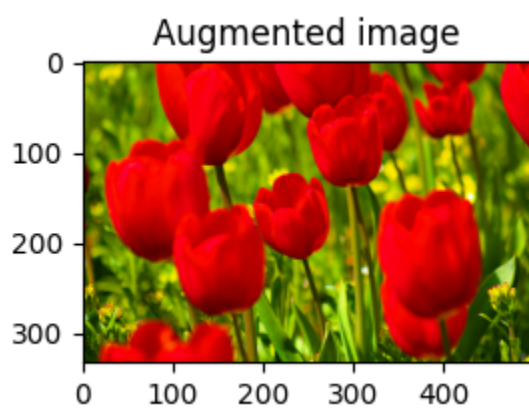
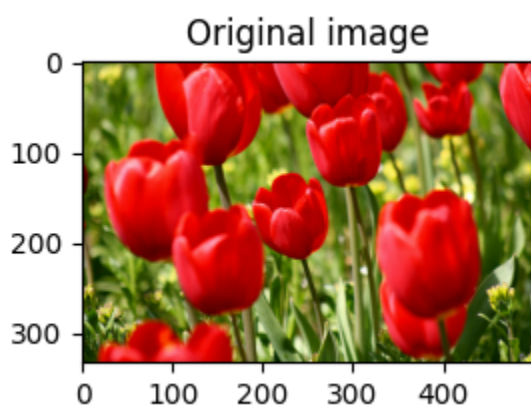
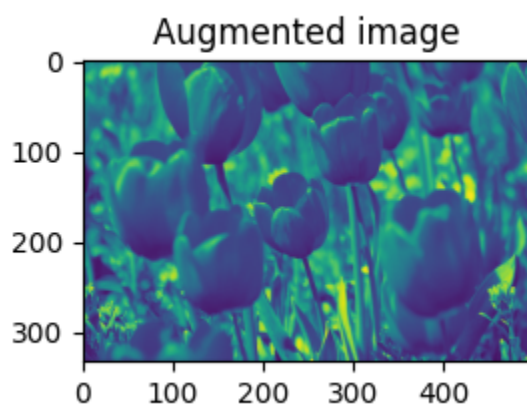
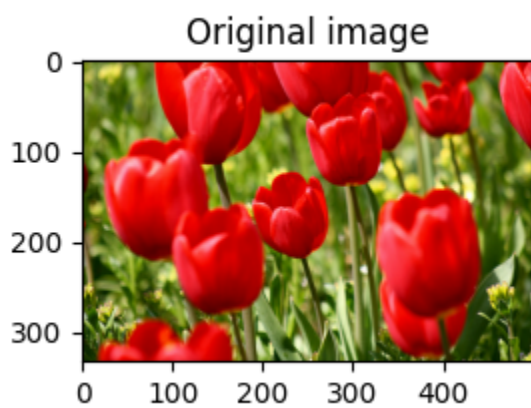
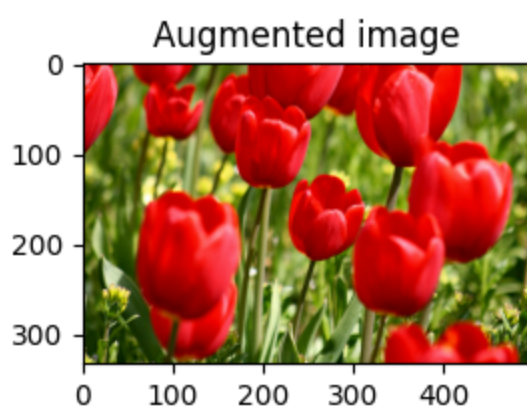
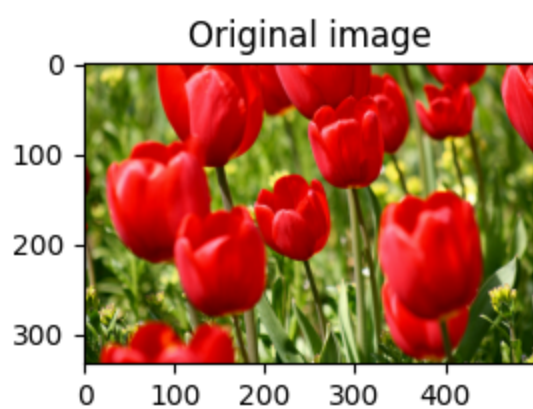


Min and max pixel values: 0.0 1.0



```
Epoch 1/5
92/92  17s 163ms/step - accuracy: 0.4101 - loss: 1.3664 - val_accuracy: 0.5504 - val_loss: 1.1178
Epoch 2/5
92/92  14s 147ms/step - accuracy: 0.5439 - loss: 1.0859 - val_accuracy: 0.6076 - val_loss: 1.0482
Epoch 3/5
92/92  14s 146ms/step - accuracy: 0.6005 - loss: 0.9859 - val_accuracy: 0.5967 - val_loss: 0.9998
Epoch 4/5
92/92  14s 144ms/step - accuracy: 0.6349 - loss: 0.9209 - val_accuracy: 0.6349 - val_loss: 0.9244
Epoch 5/5
92/92  14s 143ms/step - accuracy: 0.6512 - loss: 0.8719 - val_accuracy: 0.6376 - val_loss: 0.9525
12/12  1s 36ms/step - accuracy: 0.6458 - loss: 0.8731
Accuracy 0.6457765698432922
```

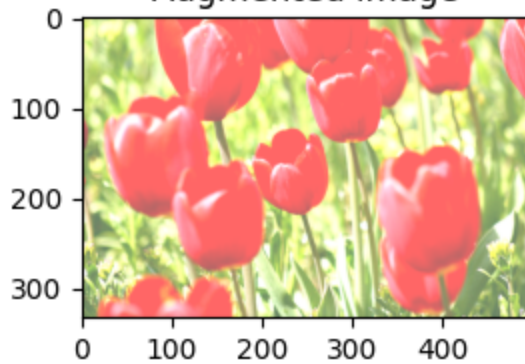




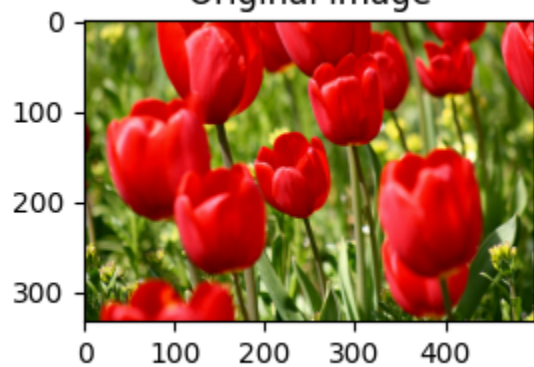
Original image



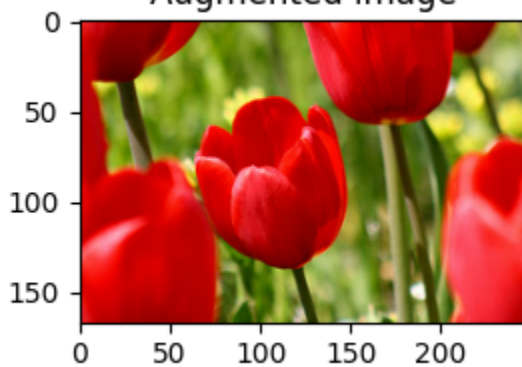
Augmented image

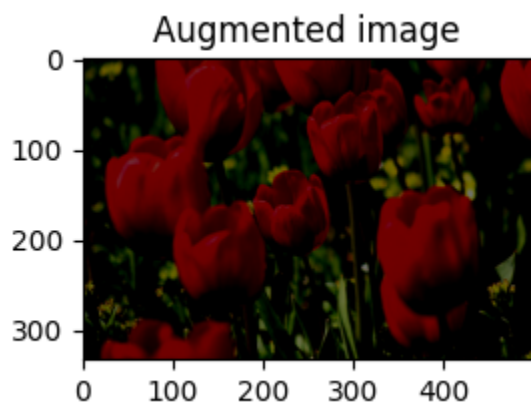
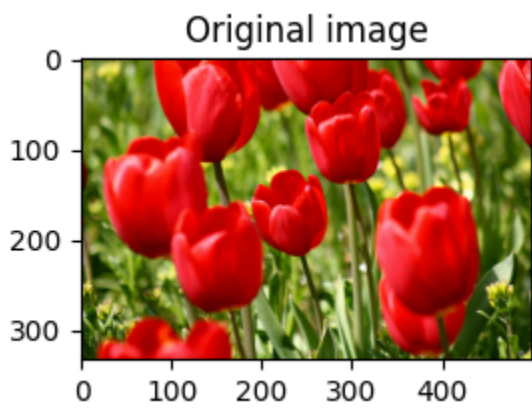
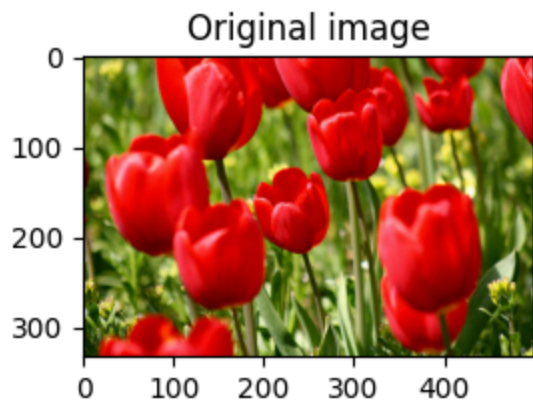


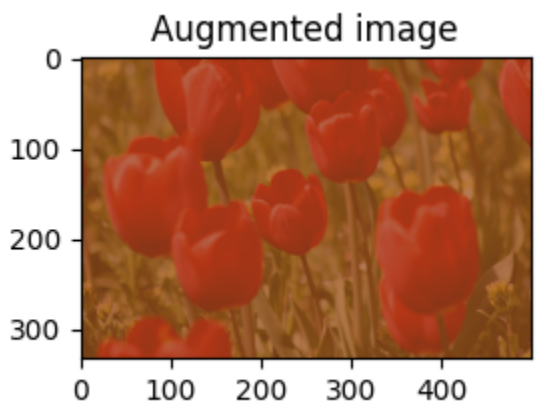
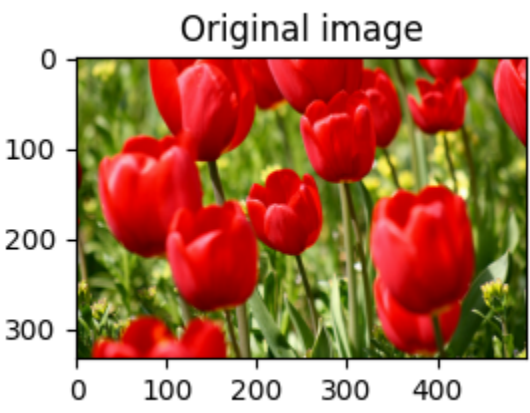
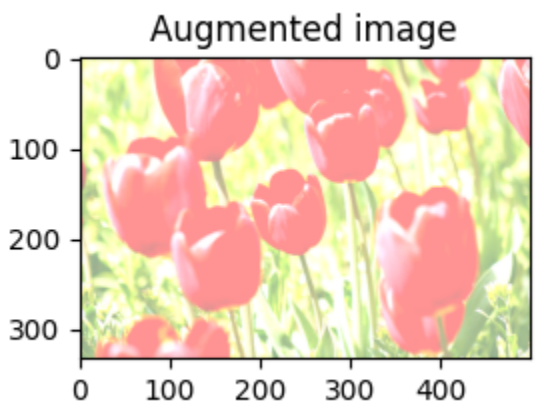
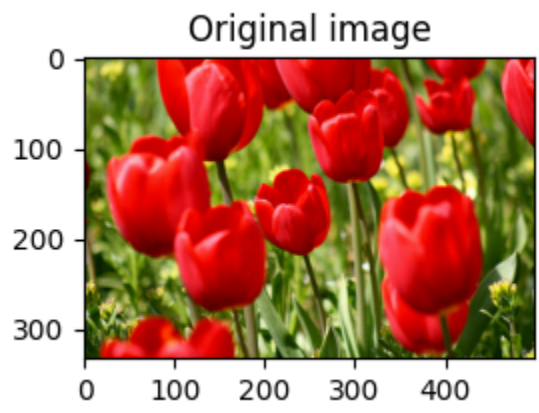
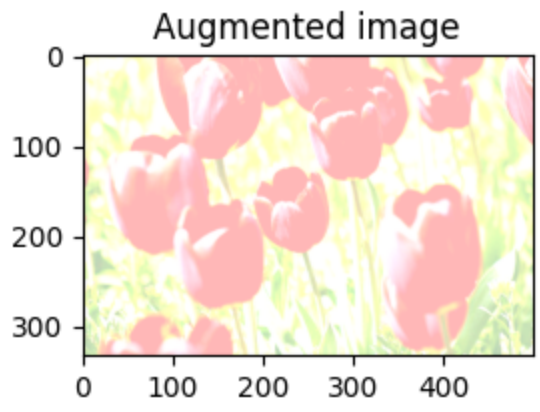
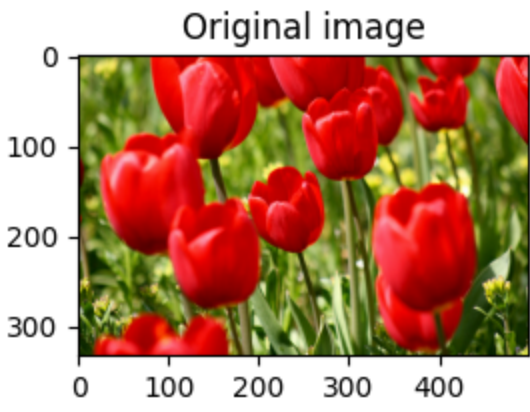
Original image

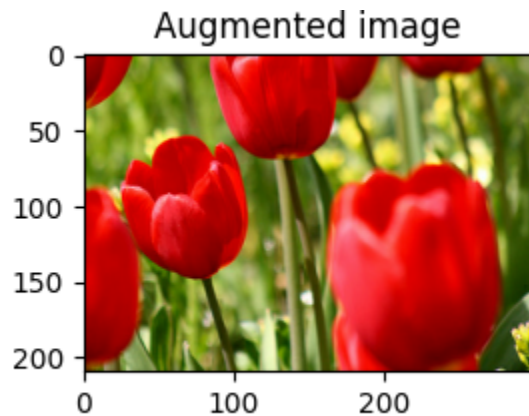
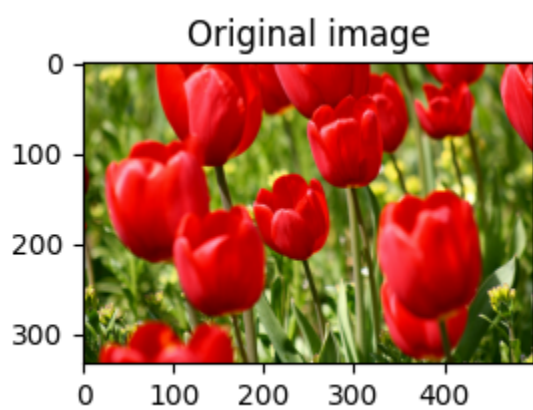
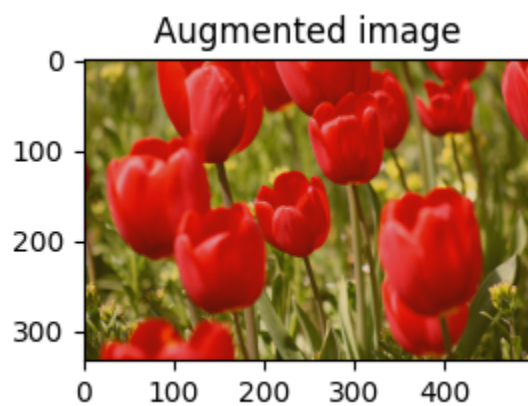
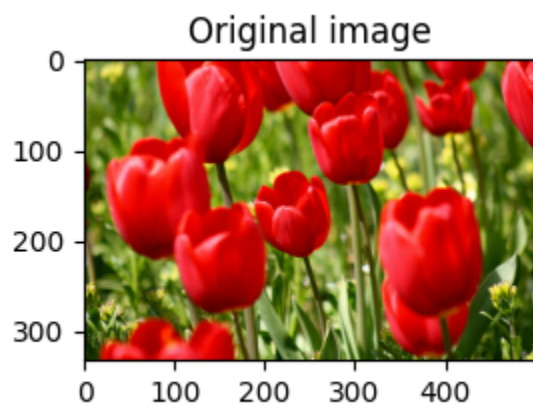
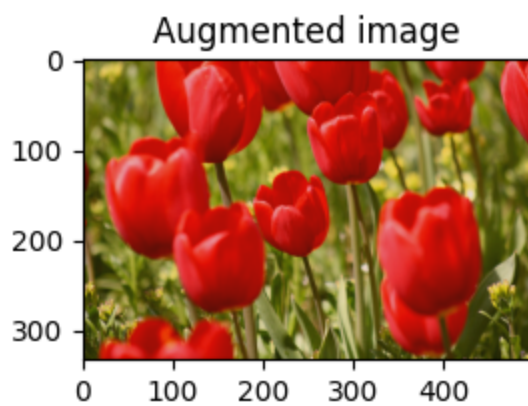
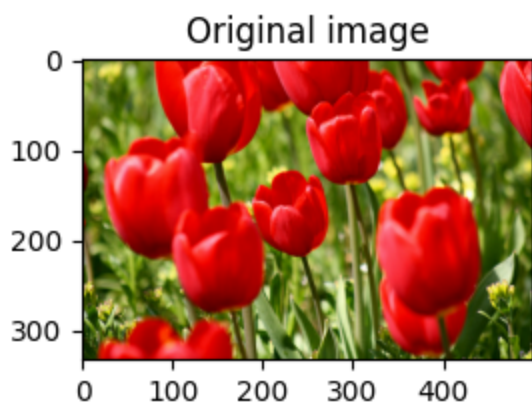


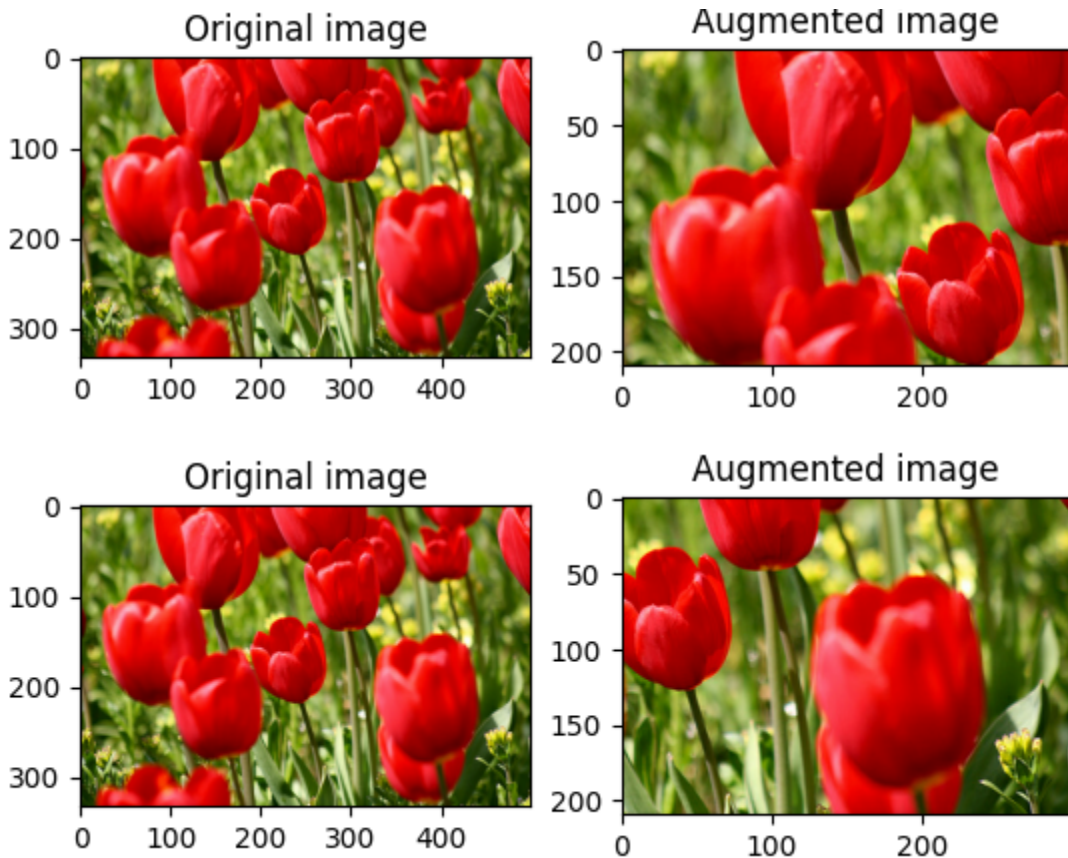
Augmented image











Accuracy 0.6457765698432922

Steps

1. Setup
 - a. Import TensorFlow and Keras preprocessing layers
2. Download a dataset
3. Resizing and rescaling
 - a. Create a small preprocessing Sequential:
 - i. Resizing and Rescaling
 - b. This standardizes image size and scales pixels to [0, 1].
4. Define a data augmentation layer
 - a. Build a Sequential with random transforms
 - b. This layer will only be active during training.

5. Choose how to use the preprocessing layers (two options)

Option A – Inside the model

- a. Start your model with:
 - i. `data_augmentation` layer
 - ii. then resizing/rescaling
 - iii. Then the rest of the CNN.
- b. Advantages: augmentation runs on GPU and is part of the saved model.

Option B – Applied to the dataset

- c. Map the augmentation to the dataset before training:
 - d. Then feed `aug_train_ds` into `model.fit()`.
6. Apply preprocessing layers to the datasets
 - a. Make sure all images get resized and rescaled
 - b. Only apply random augmentation to the training set, never the validation/test set.
7. Train a model
 - a. Build your CNN on top of the augmented and preprocessed inputs.
 - b. Compile with optimizer, loss, and metrics.
 - c. Train with `model.fit`
8. Custom data augmentation
 - a. If needed, write a custom `tf.keras.layers.Layer` or a function that:
 - i. Takes a batch of images.
 - ii. Applies custom TensorFlow ops. Returns the augmented batch.
 - b. Insert this layer into the augmentation pipeline.
 - c. Using `tf.image` directly
9. Data augmentation with `tf.image`
10. Random transformations
11. Apply augmentation to a dataset
12. Next steps
 - a. Compare training curves with and without augmentation.
 - b. Adjust the strength and types of augmentations to reduce overfitting without destroying image semantics.

Source Code

```
import matplotlib.pyplot as plt
import numpy as np
import tensorflow as tf
import tensorflow_datasets as tfds

from tensorflow.keras import layers

(train_ds, val_ds, test_ds), metadata = tfds.load(
    'tf_flowers',
    split=['train[:80%]', 'train[80%:90%]', 'train[90%:]'],
    with_info=True,
    as_supervised=True,
)

num_classes = metadata.features['label'].num_classes
print(num_classes)

get_label_name = metadata.features['label'].int2str

image, label = next(iter(train_ds))
_ = plt.imshow(image)
_ = plt.title(get_label_name(label))

IMG_SIZE = 180

resize_and_rescale = tf.keras.Sequential([
    layers.Resizing(IMG_SIZE, IMG_SIZE),
    layers.Rescaling(1./255)
])

result = resize_and_rescale(image)
_ = plt.imshow(result)

print("Min and max pixel values:", result.numpy().min(), result.numpy().max())

data_augmentation = tf.keras.Sequential([
```

```

layers.RandomFlip("horizontal_and_vertical"),
layers.RandomRotation(0.2),
])

# Add the image to a batch.
image = tf.cast(tf.expand_dims(image, 0), tf.float32)

plt.figure(figsize=(10, 10))

```

```

for i in range(9):
    augmented_image = data_augmentation(image)
    ax = plt.subplot(3, 3, i + 1)
    plt.imshow(augmented_image[0])
    plt.axis("off")
    plt.show()

```

```

batch_size = 32
AUTOTUNE = tf.data.AUTOTUNE

```

```

def prepare(ds, shuffle=False, augment=False):
    # Resize and rescale all datasets.
    ds = ds.map(lambda x, y: (resize_and_rescale(x), y),
                  num_parallel_calls=AUTOTUNE)

    if shuffle:
        ds = ds.shuffle(1000)

    # Batch all datasets.
    ds = ds.batch(batch_size)

    # Use data augmentation only on the training set.
    if augment:
        ds = ds.map(lambda x, y: (data_augmentation(x, training=True), y),
                    num_parallel_calls=AUTOTUNE)

    # Use buffered prefetching on all datasets.
    return ds.prefetch(buffer_size=AUTOTUNE)

```

```
train_ds = prepare(train_ds, shuffle=True, augment=True)
val_ds = prepare(val_ds)
test_ds = prepare(test_ds)
```

```
model = tf.keras.Sequential([
    layers.Conv2D(16, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(32, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(64, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(num_classes)
])
```

```
model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])
```

```
epochs=5
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=epochs
)
```

```
loss, acc = model.evaluate(test_ds)
print("Accuracy", acc)
```

```
def random_invert_img(x, p=0.5):
    if tf.random.uniform([]) < p:
        x = (255-x)
    else:
        x
    return x
```

```
def random_invert(factor=0.5):
    return layers.Lambda(lambda x: random_invert_img(x, factor))
```



```
random_invert = random_invert()
```

```
plt.figure(figsize=(10, 10))
```

```
for i in range(9):
```

```
    augmented_image = random_invert(image)
```

```
    ax = plt.subplot(3, 3, i + 1)
```

```
    plt.imshow(augmented_image[0].numpy().astype("uint8"))
```

```
    plt.axis("off")
```

```
    plt.show()
```

```
class RandomInvert(layers.Layer):
```

```
    def __init__(self, factor=0.5, **kwargs):
```

```
        super().__init__(**kwargs)
```

```
        self.factor = factor
```

```
    def call(self, x):
```

```
        return random_invert_img(x)
```

```
_ = plt.imshow(RandomInvert()(image)[0])
```

```
(train_ds, val_ds, test_ds), metadata = tfds.load(  
    'tf_flowers',  
    split=['train[:80%]', 'train[80%:90%]', 'train[90%:]'],  
    with_info=True,  
    as_supervised=True,  
)
```

```
image, label = next(iter(train_ds))
```

```
_ = plt.imshow(image)
```

```
_ = plt.title(get_label_name(label))
```

```
def visualize(original, augmented):
```

```
    fig = plt.figure()
```

```
    plt.subplot(1,2,1)
```

```
    plt.title('Original image')
```

```
    plt.imshow(original)
```

```
    plt.subplot(1,2,2)
```

```
    plt.title('Augmented image')
```

```
    plt.imshow(augmented)
```

```
plt.show()
```

```
flipped = tf.image.flip_left_right(image)  
visualize(image, flipped)
```

```
grayscaled = tf.image.rgb_to_grayscale(image)  
visualize(image, tf.squeeze(grayscaled))  
# _ = plt.colorbar()
```

```
saturated = tf.image.adjust_saturation(image, 3)  
visualize(image, saturated)
```

```
bright = tf.image.adjust_brightness(image, 0.4)  
visualize(image, bright)
```

```
cropped = tf.image.central_crop(image, central_fraction=0.5)  
visualize(image, cropped)
```

```
rotated = tf.image.rot90(image)  
visualize(image, rotated)
```

```
for i in range(3):  
    seed = (i, 0) # tuple of size (2,)  
    stateless_random_brightness = tf.image.stateless_random_brightness(  
        image, max_delta=0.95, seed=seed)  
    visualize(image, stateless_random_brightness)
```

```
for i in range(3):  
    seed = (i, 0) # tuple of size (2,)  
    stateless_random_contrast = tf.image.stateless_random_contrast(  
        image, lower=0.1, upper=0.9, seed=seed)  
    visualize(image, stateless_random_contrast)
```

```
for i in range(3):  
    seed = (i, 0) # tuple of size (2,)  
    stateless_random_crop = tf.image.stateless_random_crop(  
        image, size=[210, 300, 3], seed=seed)  
    visualize(image, stateless_random_crop)
```

```
(train_datasets, val_ds, test_ds), metadata = tfds.load(
    'tf_flowers',
    split=['train[:80%]', 'train[80%:90%]', 'train[90%:]'],
    with_info=True,
    as_supervised=True,
)
```

```
def resize_and_rescale(image, label):
    image = tf.cast(image, tf.float32)
    image = tf.image.resize(image, [IMG_SIZE, IMG_SIZE])
    image = (image / 255.0)
    return image, label
```

```
def augment(image_label, seed):
    image, label = image_label
    image, label = resize_and_rescale(image, label)
    image = tf.image.resize_with_crop_or_pad(image, IMG_SIZE + 6, IMG_SIZE + 6)
    # Make a new seed.
    new_seed = tf.random.split(seed, num=1)[0, :]
    # Random crop back to the original size.
    image = tf.image.stateless_random_crop(
        image, size=[IMG_SIZE, IMG_SIZE, 3], seed=seed)
    # Random brightness.
    image = tf.image.stateless_random_brightness(
        image, max_delta=0.5, seed=new_seed)
    image = tf.clip_by_value(image, 0, 1)
    return image, label
```

```
# Create a generator.
rng = tf.random.Generator.from_seed(123, alg='philox')
```

```
# Create a wrapper function for updating seeds.
```

```
def f(x, y):
    seed = rng.make_seeds(1)[: , 0]
    image, label = augment((x, y), seed)
    return image, label
```

```
train_ds = (
    train_datasets
    .shuffle(1000)
```

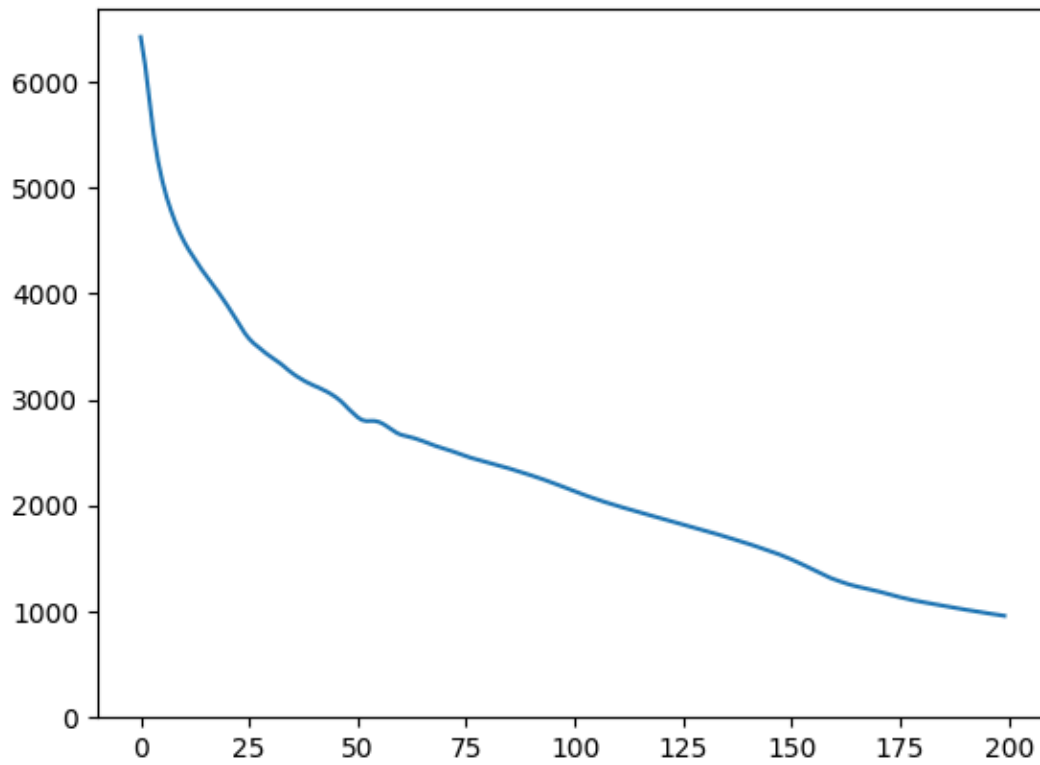
```
.map(f, num_parallel_calls=AUTOTUNE)  
.batch(batch_size)  
.prefetch(AUTOTUNE)  
)
```

```
val_ds = (  
    val_ds  
    .map(resize_and_rescale, num_parallel_calls=AUTOTUNE)  
    .batch(batch_size)  
    .prefetch(AUTOTUNE)  
)
```

```
test_ds = (  
    test_ds  
    .map(resize_and_rescale, num_parallel_calls=AUTOTUNE)  
    .batch(batch_size)  
    .prefetch(AUTOTUNE)  
)
```

Generate Artificial Faces with CelebA Progressive GAN Model

Output



Steps

1. Setup
 - a. Import needed libraries
 - b. Load the ProGAN CelebA model from TensorFlow Hub and store it in a variable.
 - c. Define helper functions to:
 - i. Generate images from latent vectors using the ProGAN model.
 - ii. Display or save images.
 - iii. Create and save GIF animations.
2. Latent space interpolation
 - a. Sample two random latent vectors v_1 and v_2 from a normal distribution
 - b. Define a function that interpolates between v_1 and v_2 on the hypersphere
 - c. Pass the interpolated latent vectors into the ProGAN model to generate a sequence of face images.
 - d. Visualize the interpolation as:
 - i. An animated GIF showing the face morphing from v_1 to v_2 .
3. Latent space interpolation – more experiments
 - a. Repeat the interpolation with different random pairs of latent vectors.
 - b. Change the number of interpolation steps to get smoother or faster transitions.
 - c. Observe how facial attributes change smoothly along the interpolation path.
4. Finding the closest vector in latent space
 - a. Choose a target face image
 - b. Sample many random latent vectors and generate a face for each one.
 - c. For each generated face, compute a distance to the target image.
5. Visualize the closest match
 - a. Generate a face from the closest latent vector.
 - b. Create a side-by-side image that shows:
 - i. The closest generated face
 - ii. The target face image.
 - c. Compare how similar they look and how well the GAN approximates the target.

Source Code

```
from absl import logging

import imageio
import PIL.Image
import matplotlib.pyplot as plt
import numpy as np

import tensorflow as tf
tf.random.set_seed(0)

import tensorflow_hub as hub
from tensorflow_docs.vis import embed
import time

try:
    from google.colab import files
except ImportError:
    pass

from IPython import display
from skimage import transform

# We could retrieve this value from module.get_input_shapes() if we didn't know
# beforehand which module we will be using.
latent_dim = 512

# Interpolates between two vectors that are non-zero and don't both lie on a
# line going through origin. First normalizes v2 to have the same norm as v1.
# Then interpolates between the two vectors on the hypersphere.
def interpolate_hypersphere(v1, v2, num_steps):
    v1_norm = tf.norm(v1)
    v2_norm = tf.norm(v2)
    v2_normalized = v2 * (v1_norm / v2_norm)

    vectors = []
    for step in range(num_steps):
        interpolated = v1 + (v2_normalized - v1) * step / (num_steps - 1)
        interpolated_norm = tf.norm(interpolated)
```

```

    interpolated_normalized = interpolated * (v1_norm / interpolated_norm)
    vectors.append(interpolated_normalized)
    return tf.stack(vectors)

# Simple way to display an image.
def display_image(image):
    image = tf.constant(image)
    image = tf.image.convert_image_dtype(image, tf.uint8)
    return PIL.Image.fromarray(image.numpy())

# Given a set of images, show an animation.
def animate(images):
    images = np.array(images)
    converted_images = np.clip(images * 255, 0, 255).astype(np.uint8)
    imageio.mimsave('./animation.gif', converted_images)
    return embed.embed_file('./animation.gif')

logging.set_verbosity(logging.ERROR)

progan = hub.load("https://tfhub.dev/google/progan-128/1").signatures['default']

def interpolate_between_vectors():
    v1 = tf.random.normal([latent_dim])
    v2 = tf.random.normal([latent_dim])

    # Creates a tensor with 25 steps of interpolation between v1 and v2.
    vectors = interpolate_hypersphere(v1, v2, 50)

    # Uses module to generate images from the latent space.
    interpolated_images = progan(vectors)['default']

    return interpolated_images

interpolated_images = interpolate_between_vectors()
animate(interpolated_images)

image_from_module_space = True # @param { isTemplate:true, type:"boolean" }

def get_module_space_image():
    vector = tf.random.normal([1, latent_dim])

```



```

images = progan(vector)['default'][0]
return images

def upload_image():
    uploaded = files.upload()
    image = imageio.imread(uploaded[list(uploaded.keys())[0]])
    return transform.resize(image, [128, 128])

if image_from_module_space:
    target_image = get_module_space_image()
else:
    target_image = upload_image()

display_image(target_image)

tf.random.set_seed(42)
initial_vector = tf.random.normal([1, latent_dim])

display_image(progan(initial_vector)['default'][0])

def find_closest_latent_vector(initial_vector, num_optimization_steps,
                               steps_per_image):
    images = []
    losses = []

    vector = tf.Variable(initial_vector)
    optimizer = tf.optimizers.Adam(learning_rate=0.01)
    loss_fn = tf.losses.MeanAbsoluteError(reduction="sum")

    for step in range(num_optimization_steps):
        if (step % 100) == 0:
            print()
            print('.', end="")
            with tf.GradientTape() as tape:
                image = progan(vector.read_value())['default'][0]
                if (step % steps_per_image) == 0:
                    images.append(image.numpy())
                target_image_difference = loss_fn(image, target_image[:, :, :3])
            # The latent vectors were sampled from a normal distribution. We can get
            # more realistic images if we regularize the length of the latent vector to

```

```

# the average length of vector from this distribution.
regularizer = tf.abs(tf.norm(vector) - np.sqrt(latent_dim))

loss = target_image_difference + regularizer
losses.append(loss.numpy())
grads = tape.gradient(loss, [vector])
optimizer.apply_gradients(zip(grads, [vector]))

return images, losses

num_optimization_steps=200
steps_per_image=5
images, loss = find_closest_latent_vector(initial_vector, num_optimization_steps,
steps_per_image)

plt.plot(loss)
plt.ylim([0,max(plt.ylim())])
plt.show()

animate(np.stack(images))

display_image(np.concatenate([images[-1], target_image], axis=1))

```

Bibliography

TensorFlow. (2024, August 16). *Convolutional neural network (CNN)*. TensorFlow Core Tutorials. Retrieved November 14, 2025, from <https://www.tensorflow.org/tutorials/images/cnn>

TensorFlow. (2024, April 3). *Image classification*. TensorFlow Core Tutorials. Retrieved November 14, 2025, from <https://www.tensorflow.org/tutorials/images/classification>

TensorFlow. (2024, July 19). *Data augmentation*. TensorFlow Core Tutorials. Retrieved November 15, 2025, from https://www.tensorflow.org/tutorials/images/data_augmentation

Google. (n.d.). *Progressive GAN 128x128 (CelebA)* [TensorFlow Hub model]. TensorFlow Hub. Retrieved November 16, 2025, from <https://www.tensorflow.org>